HOSPITAL MANAGEMENT SYSTEM

Software Requirements Specification (SRS)

Version 1.0

Project Title	Hospital Management System
Version	1.0
Project Team	Ukkasha Ahmed, Yesaullah Sheikh
Submission Date	May 2026

Document History

Version	Name of Person	Date	Description of Change
1.0	Yesaullah	April 2026	Document Created
1.0	Ukkasha	April 2026	Functional Requirements Added
1.0	Yesaullah	April 2026	Non-Functional Requirements Added
1.0	Ukkasha	April 2026	Use Cases Added

Distribution List

Name	Role
Ms Sania Urooj	Supervisor
Yesaullah Sheikh	Developer
Ukkasha Ahmed	Developer

Document Sign-Off

Version	Sign-off Authority	Sign-off Date
1.0	Ms Sania Urooj	

1. Introduction

1.1 Purpose of Document

This Software Requirements Specification (SRS) document defines and documents the detailed requirements for the Hospital Management System (HMS). The purpose of this document is to provide a comprehensive description of the system to be developed, serving as a foundation for system design, implementation, testing, and project planning. It captures functional and non-functional requirements, system constraints, and interface specifications to ensure a shared understanding among all stakeholders and the development team.

1.2 Intended Audience

This document is intended for the following audiences:

- Project Supervisor and Co-Supervisor: For review, guidance, and approval of requirements
- Development Team (Group Members): As a reference guide during system design and implementation
- Hospital Staff (Stakeholders): To validate that the system meets operational needs
- Quality Assurance Engineers: For preparing test plans and test cases
- Future Maintainers: To understand the system scope and functionality

1.3 Abbreviations

Abbreviation	Full Form
HMS	Hospital Management System
SRS	Software Requirements Specification
FR	Functional Requirement
NFR	Non-Functional Requirement
UI	User Interface
DB	Database
API	Application Programming Interface
ORM	Object-Relational Mapping
RBAC	Role-Based Access Control
CRUD	Create, Read, Update, Delete
JWT	JSON Web Token
WTF	Web Template Forms (Flask-WTF)

1.4 Document Convention

This document uses the following formatting conventions:

- Font: Times New Roman
- Body Text Size: 12pt (24 half-points)
- Headings: Bold Times New Roman, sizes 14pt (H1), 12pt (H2), 11pt (H3)
- Requirement identifiers follow the pattern: FR-XX for functional requirements and NFR-XX for non-functional requirements
- Use case identifiers follow the pattern: UC-XX
- Table headers use a blue background (#2E5090) with white text for visual clarity

2. Overall System Description

2.1 Project Background

Healthcare facilities rely on the coordinated efforts of multiple staff roles including doctors, nurses, receptionists, pharmacists, and store managers. In many hospitals, these workflows are still partially manual or fragmented across disconnected tools, leading to inefficiencies in appointment scheduling, patient admission, prescription handling, billing, and inventory management. This results in delayed care delivery, billing errors, stock shortages, and poor visibility across departments.

The Hospital Management System (HMS) is developed as a Final Year Project (FYP) to address these operational challenges by providing a unified, web-based platform that digitises and integrates the core workflows of a hospital. The system is built using Flask (Python), SQLAlchemy ORM, and a SQLite database, with a Jinja2 server-rendered frontend.

2.2 Project Scope

The HMS provides a role-based web application for five user types: Doctor, Nurse, Receptionist, Store Manager, and Patient. The system covers the following functional areas:

- User registration, authentication, and role-based access control
- Patient registration, profile management, and medical record maintenance
- Appointment scheduling, management, and status tracking
- Inpatient admission, room assignment, and discharge management
- Nurse assignment to patients and real-time patient status recording (vitals)
- Doctor prescription issuance and medication management
- Hospital inventory management including medications, equipment, and supplies
- Supplier management and purchase order lifecycle tracking
- Bill generation and payment tracking for both outpatient and inpatient cases

2.3 Not In Scope

The following functionalities are explicitly outside the scope of this project:

- Mobile application development (iOS/Android native apps)
- Integration with external laboratory or radiology information systems
- Real-time telemedicine or video consultation features
- Insurance claim processing and third-party payment gateway integration
- Multi-hospital or multi-branch deployment and management
- Advanced analytics dashboards and AI-driven diagnostic support
- Electronic Health Record (EHR) interoperability with national health databases

2.4 Project Objectives

The primary objectives of the Hospital Management System are:

- To digitise and centralise hospital operations into a single, cohesive web platform
- To enforce role-based access control ensuring each staff member interacts only with their designated functions
- To streamline the patient journey from registration and appointment booking through admission, treatment, and discharge
- To automate medication dispensing records and ensure inventory stock levels are accurately maintained
- To generate accurate, itemised bills for patients covering room charges, medication costs, and medical procedure fees
- To provide nurses with real-time tools for monitoring and recording patient vitals and condition status
- To facilitate the procurement cycle by managing suppliers, purchase orders, and order status

2.5 Stakeholders

Stakeholder	Role	Interest / Involvement
Doctor	End User	Manages appointments, writes prescriptions, views patient history
Nurse	End User	Manages patient assignments, records vitals, monitors room status
Receptionist	End User	Registers patients, schedules appointments, handles admission and billing
Store Manager	End User	Manages inventory, suppliers, and purchase orders
Patient	End User / Subject	Receives care; their data is the central subject of the system
Hospital Administrator	Business Stakeholder	Oversees operations; benefits from improved efficiency
Development Team	Technical Stakeholder	Designs, builds, and maintains the system
Project Supervisor	Academic Stakeholder	Reviews and guides the project for academic compliance

2.6 Operating Environment

The HMS is a server-side web application designed to operate in the following environment:

- Server: Python 3.x runtime with Flask development server or Gunicorn for production
- Database: SQLite (development and FYP scope); switchable to MySQL/PostgreSQL via DATABASE_URL environment variable

- Client: Any modern web browser (Chrome, Firefox, Edge, Safari) on desktop or laptop computers
- Operating System: Cross-platform; server supports Linux and Windows; client requires no installation
- Network: Local area network (LAN) or internet connectivity for accessing the web interface
- Dependencies: Flask ecosystem libraries as listed in requirements.txt

2.7 System Constraints

The following constraints apply to the system:

- Software Constraints: The system must operate within the Flask and SQLAlchemy ecosystem. SQLite imposes a single-writer concurrency limitation suitable for development but not for high-concurrency production deployments.
- Hardware Constraints: The server requires a minimum of 2 GB RAM and 10 GB storage. Client machines require only a browser and network access.
- Security Constraints: Passwords must be hashed using bcrypt. All routes must enforce authentication via Flask-Login. JWT tokens must be issued with expiry.
- Academic Constraints: The project must be completed within a 2-month timeline with two developers.
- User Constraints: Hospital staff are assumed to have basic computer literacy. The interface must be intuitive and accessible through standard browsers without requiring plugins.
- Language Constraints: The system interface will be in English.

2.8 Assumptions & Dependencies

Assumptions:

- Each user has a valid email address and a unique username for registration
- Room numbers are pre-seeded in the database before the system goes live (via seed_rooms.py)
- Medications and inventory items are initially loaded by the store manager before prescriptions can reference them
- Users with the receptionist role are trained to handle both patient admission and billing workflows

Dependencies:

- Flask and all listed dependencies in requirements.txt must be installed and available
- Python 3.8 or higher must be present on the server
- The DATABASE_URL environment variable, if set, must point to a valid relational database URI
- Flask-Migrate (Alembic) must be used for any schema changes to preserve database state

3. External Interface Requirements

The following context describes the external interfaces at a high level. The HMS interacts with the browser client, the underlying database, and optionally with an SMTP mail server for future notification features.

3.1 Hardware Interfaces

The HMS is a web-based application and does not directly interface with specialised hardware. However, the following hardware interactions are relevant:

- Client Devices: Desktop computers and laptops used by hospital staff to access the application through a browser. No special hardware peripherals are required.
- Server Hardware: The application server requires standard commodity hardware with sufficient CPU and RAM to handle concurrent Flask requests. Minimum specifications: 2-core CPU, 2 GB RAM, 10 GB storage.
- Network Hardware: A standard LAN router/switch is required to enable connectivity between the application server and client workstations within the hospital network.

3.2 Software Interfaces

The HMS interfaces with the following software components:

- Python Runtime (3.8+): The server-side execution environment for the Flask application.
- Flask (v2.3.3): The core web framework handling routing, request processing, and response generation.
- SQLAlchemy (v2.0.23) with Flask-SQLAlchemy (v3.1.1): ORM layer for database operations, supporting both SQLite and MySQL/PostgreSQL.
- SQLite / MySQL / PostgreSQL: The relational database management system storing all persistent data. SQLite is used during development.
- Flask-Login (v0.6.2): Manages user session lifecycle, authentication state, and protected route enforcement.
- Flask-WTF with WTForms (v1.2.1 / v3.1.1): Form handling, CSRF protection, and input validation.
- Flask-Bcrypt (v1.0.1): Password hashing and verification.
- Flask-JWT-Extended (v4.5.3): JSON Web Token issuance and validation for API authentication.
- Flask-Migrate / Alembic (v4.0.5): Database schema migration management.
- Jinja2 (v3.1.2): Server-side HTML template rendering engine.
- Gunicorn (v21.2.0): Production-grade WSGI HTTP server for deployment.

3.3 Communications Interfaces

The HMS uses the following communication interfaces:

- HTTP/HTTPS Protocol: All communication between the client browser and the Flask server occurs over HTTP (development) or HTTPS (production) using standard TCP/IP. Forms submit data via HTTP POST; page navigation uses HTTP GET.
- CORS: Flask-CORS is configured to allow cross-origin requests, enabling potential future API clients.
- Session Cookies: Flask-Login uses secure, signed session cookies to maintain authenticated user sessions. Cookies should be configured with HttpOnly and Secure flags in production.
- CSRF Tokens: All form submissions include CSRF tokens managed by Flask-WTF to prevent cross-site request forgery.
- Local File System: The application reads environment variables from a .env file using python-dotenv for configuration (SECRET_KEY, DATABASE_URL, JWT_SECRET_KEY).

4. Functional Requirements

4.1 Functional Hierarchy

The following hierarchy describes the main modules and their sub-functions:

- FR-01: Authentication & User Management
  - FR-01.1: User Registration (Doctor, Nurse, Receptionist, Store Manager)
  - FR-01.2: User Login and Logout
  - FR-01.3: Role-Based Access Control (RBAC)
  - FR-01.4: Profile Creation and Editing per Role

- FR-02: Patient Management
  - FR-02.1: Patient Registration by Receptionist
  - FR-02.2: Patient Profile Viewing and Editing
  - FR-02.3: Medical Record Creation and Retrieval
  - FR-02.4: Patient Search

- FR-03: Appointment Management
  - FR-03.1: Schedule Appointment (Receptionist / Patient)
  - FR-03.2: Edit and Cancel Appointments
  - FR-03.3: View Appointment Details
  - FR-03.4: Doctor Schedule Viewing

- FR-04: Inpatient Management
  - FR-04.1: Admit Patient to Room
  - FR-04.2: Assign Nurse to Admitted Patient
  - FR-04.3: Discharge Patient
  - FR-04.4: Room Status Management (Available, Occupied, Maintenance)

- FR-05: Nurse Functions
  - FR-05.1: View Assigned Patients
  - FR-05.2: Record Patient Vitals and Status Updates
  - FR-05.3: Create and End Patient Assignments
  - FR-05.4: View Room Overview

- FR-06: Doctor Functions
  - FR-06.1: View and Manage Appointments
  - FR-06.2: View Patient Details and Medical History
  - FR-06.3: Create Prescriptions
  - FR-06.4: Update Appointment Status (Scheduled, Completed, Cancelled)

- FR-07: Prescription & Medication Management
  - FR-07.1: Create Prescription with Medication Items
  - FR-07.2: Dispense Medication and Deduct Stock
  - FR-07.3: View Prescription History

- FR-08: Billing Management
  - FR-08.1: Generate Bill for Inpatient Admission
  - FR-08.2: Generate Bill for Outpatient Appointment
  - FR-08.3: Calculate Total from Room, Medical, Medication, and Other Charges
  - FR-08.4: Record Payment Status (Pending, Paid, Partially Paid)

- FR-09: Inventory & Store Management
  - FR-09.1: Add, Edit, View Inventory Items
  - FR-09.2: Manage Suppliers
  - FR-09.3: Create Purchase Orders and Add Order Items
  - FR-09.4: Update Order Status (Pending, Delivered, Cancelled)
  - FR-09.5: Update Inventory Stock upon Order Delivery

4.2 Use Cases

4.2.1 UC-01: User Login

UC-01: User Login		
Use Case Id	UC-01: User Login	
Actors	Any registered user (Doctor, Nurse, Receptionist, Store Manager)	
Feature	FR-01: Authentication & User Management	
Pre-condition	User has a registered account with valid credentials.	
Scenarios		
Step#	**Action**	**Software Reaction**
1.	User navigates to the login page.	System displays the login form with email and password fields.
2.	User enters their registered email address and password.	System validates that both fields are filled.
3.	User clicks the Login button.	System verifies credentials against the database using bcrypt hash comparison.

4.	Credentials are valid.	System creates a user session via Flask-Login and redirects to the home/dashboard page.
Alternate Scenarios		
3a: If credentials are invalid, the system displays: 'Login unsuccessful. Please check email and password.' 3b: If the account does not exist, the same generic error is displayed to prevent user enumeration. 2a: If either field is empty, form validation prevents submission and highlights the empty field.		
Post Conditions		
Step#	**Description**	
1.	User is authenticated and a session cookie is set in the browser.	
2.	User is redirected to the main home page.	
3.	The navigation menu updates to reflect the logged-in user's role.	
Use Case Cross-referenced	UC-02 (Role-Based Dashboard Routing)	

4.2.2 UC-02: Patient Registration by Receptionist

UC-02: Patient Registration		
Use Case Id	UC-02: Patient Registration	
Actors	Receptionist	
Feature	FR-02: Patient Management	
Pre-condition	Receptionist is logged in and has been assigned the receptionist role.	
Scenarios		
Step#	**Action**	**Software Reaction**
1.	Receptionist navigates to 'Register New Patient'.	System displays the patient registration form.
2.	Receptionist fills in: first name, last name, date of birth, gender, blood group, contact number, email, address, and emergency contact.	System validates all required fields in real time using WTForms validators.
3.	Receptionist submits the form.	System saves the new Patient record to the database with the current timestamp as registration_date.
4.	System creates the patient record.	System displays a success message and redirects to the patient's profile page.

Alternate Scenarios	
3a: If required fields are missing or invalid (e.g., invalid email format), the form is re-rendered with validation error messages. 3b: If a duplicate contact number is detected, the system alerts the receptionist.	
Post Conditions	
Step#	**Description**
1.	A new Patient record is persisted in the database.
2.	The patient appears in the patient list and can be referenced in appointments, admissions, and billing.
Use Case Cross-referenced	UC-03 (Schedule Appointment), UC-05 (Admit Patient)

4.2.3 UC-03: Schedule Appointment

UC-03: Schedule Appointment		
Use Case Id	UC-03: Schedule Appointment	
Actors	Receptionist / Patient	
Feature	FR-03: Appointment Management	
Pre-condition	The patient exists in the system. At least one doctor profile exists.	
Scenarios		
Step#	**Action**	**Software Reaction**
1.	Actor navigates to 'Schedule Appointment'.	System displays the appointment scheduling form with a list of available doctors.
2.	Actor selects a doctor, enters the appointment date/time, and provides a purpose description.	System pre-validates the selected date is in the future.
3.	Actor submits the form.	System creates an Appointment record with status 'scheduled' and links the patient and doctor IDs.
4.	Appointment is saved.	System confirms the booking and displays appointment details.
Alternate Scenarios		
2a: If the selected date/time is in the past, the form rejects submission with an error. 2b: If the selected doctor has a conflicting appointment at the same time, the system warns the receptionist.		

Post Conditions	
Step#	**Description**
1.	A new Appointment record is created with status 'scheduled'.
2.	The appointment appears in both the doctor's and patient's appointment views.
Use Case Cross-referenced	UC-01 (Login), UC-05 (Admit Patient), UC-07 (Generate Bill)

4.2.4 UC-04: Admit Patient (Inpatient)

UC-04: Admit Patient		
Use Case Id	UC-04: Admit Patient	
Actors	Receptionist	
Feature	FR-04: Inpatient Management	
Pre-condition	Patient is registered. At least one room with status 'available' exists. At least one nurse is registered.	
Scenarios		
Step#	**Action**	**Software Reaction**
1.	Receptionist opens the patient profile and clicks 'Admit Patient'.	System displays the admission form with available rooms, doctors, and nurses.
2.	Receptionist selects a room, assigning doctor, assigning nurse, and enters the reason for admission.	System validates all required fields.
3.	Receptionist submits the admission form.	System creates an Admission record, sets the room status to 'occupied', and creates a NurseAssignment record.
4.	Admission is confirmed.	System displays a success message and updates the patient profile with admission status.
Alternate Scenarios		
2a: If no rooms are available, the system displays 'No rooms currently available' and prevents submission. 3a: If a database error occurs during room status update, the transaction is rolled back.		
Post Conditions		
Step#	**Description**	
1.	An Admission record is persisted with status 'admitted'.	

2.	The selected room's status is set to 'occupied'.
3.	A NurseAssignment record is created linking the nurse to the patient.
Use Case Cross-referenced	UC-06 (Discharge Patient), UC-07 (Generate Bill)

4.2.5 UC-05: Create Prescription

UC-05: Create Prescription		
Use Case Id	UC-05: Create Prescription	
Actors	Doctor	
Feature	FR-07: Prescription & Medication Management	
Pre-condition	Doctor is logged in. Patient has an appointment with this doctor. Medications exist in the system.	
Scenarios		
Step#	**Action**	**Software Reaction**
1.	Doctor navigates to the patient details page and clicks 'Create Prescription'.	System displays the prescription form with a list of available medications.
2.	Doctor enters general prescription notes and adds one or more medication items specifying medication, dosage, frequency, duration, and quantity.	System validates all medication fields.
3.	Doctor submits the prescription.	System saves the Prescription and PrescriptionMedication records to the database.
4.	Prescription is saved.	System confirms creation and displays the prescription on the patient details page.
Alternate Scenarios		
2a: If no medications exist in the system, the doctor sees an empty medication list and cannot add items. 3a: If a required field (dosage, quantity) is missing, the form re-renders with validation errors.		
Post Conditions		
Step#	**Description**	
1.	A Prescription record is created linked to the doctor and patient.	
2.	One or more PrescriptionMedication records are created, ready for dispensing.	

3.	Stock is NOT deducted at prescription creation; deduction occurs on dispensing.
Use Case Cross-referenced	UC-06 (Dispense Medication)

4.2.6 UC-06: Generate Bill

UC-06: Generate Bill		
Use Case Id	UC-06: Generate Bill	
Actors	Receptionist	
Feature	FR-08: Billing Management	
Pre-condition	An Admission or Appointment record exists for the patient. Receptionist is logged in.	
Scenarios		
Step#	**Action**	**Software Reaction**
1.	Receptionist navigates to the admission or appointment details page and clicks 'Generate Bill'.	System displays the bill generation form pre-populated with patient and admission/appointment details.
2.	Receptionist enters room charges (for inpatients), medication charges, medical/procedure charges, and other charges.	System displays running sub-totals for each charge category.
3.	Receptionist submits the bill.	System calls calculate_total(), sums all charge fields, and persists the Bill record.
4.	Bill is saved.	System displays the final bill with total amount and payment status 'pending'.
Alternate Scenarios		
3a: If any charge field contains a negative value, validation prevents submission. 4a: If payment is collected immediately, the receptionist can set payment_status to 'paid' before saving.		
Post Conditions		
Step#	**Description**	
1.	A Bill record is created linked to the Admission or Appointment.	
2.	Total amount is calculated and stored.	
3.	Payment status is set to 'pending' by default unless changed by the receptionist.	

Use Case Cross-referenced	UC-04 (Admit Patient), UC-07 (Discharge Patient)

5. Non-Functional Requirements

5.1 Performance Requirements

ID	Requirement	Target
NFR-01	Page Response Time: All standard page requests must complete and render in the browser within 3 seconds under normal load.	< 3 seconds
NFR-02	Database Query Performance: Queries against common tables (Patients, Appointments, Admissions) must execute within 500ms.	< 500ms
NFR-03	Concurrent Users: The system must support at least 20 concurrent authenticated users without performance degradation.	20+ users
NFR-04	Session Scalability: Flask-Login sessions must be managed efficiently; session storage must not grow unbounded.	Stateless cookies
NFR-05	Bill Calculation: The bill total calculation (calculate_total()) must execute synchronously within 100ms.	< 100ms

5.2 Safety Requirements

ID	Requirement
NFR-06	Data Integrity: All database writes that involve multiple related records (e.g., admission + room status update) must use atomic transactions with rollback on failure to prevent partial data corruption.
NFR-07	Patient Data Protection: The system must not expose patient data (medical records, prescriptions, billing) to users of other roles. RBAC guards must be applied on every route.
NFR-08	Medication Dispensing Safety: The PrescriptionMedication.dispense() method must verify sufficient stock before deducting. If stock is insufficient, the operation must fail gracefully with an appropriate message.
NFR-09	Room State Transitions: Room status must only transition through valid states (available -> occupied -> available; available -> maintenance -> available) as enforced by Room model methods.
NFR-10	Input Validation: All user inputs submitted via forms must be validated on the server side using WTForms validators to prevent malformed data from reaching the database.

5.3 Security Requirements

ID	Requirement
NFR-11	Password Hashing: All user passwords must be hashed using Flask-Bcrypt before storage. Plain-text passwords must never be stored or logged.
NFR-12	Authentication Enforcement: All routes except /login, /register, and / (home) must require authentication via the @login_required decorator.

ID	Requirement
NFR-13	Role-Based Access Control: Every protected route must verify current_user.role matches the expected role and redirect unauthorised users to the home page with an error flash message.
NFR-14	CSRF Protection: All HTML forms must include a CSRF token generated and validated by Flask-WTF to protect against cross-site request forgery attacks.
NFR-15	Session Security: The SECRET_KEY must be sourced from environment variables and must be a high-entropy random string in production. Default development keys must not be used in production.
NFR-16	JWT Security: JWT tokens issued by Flask-JWT-Extended must have an expiry time. The JWT_SECRET_KEY must be set via environment variable and not hardcoded.
NFR-17	SQL Injection Prevention: All database queries must use SQLAlchemy ORM parameterised queries. Raw SQL strings must not be used.

5.4 User Documentation

The following documentation components will be delivered alongside the software:

- User Manual: A role-specific user manual covering each dashboard, form, and workflow for Doctor, Nurse, Receptionist, and Store Manager roles with annotated screenshots.
- System Setup Guide: A technical guide covering environment setup, dependency installation (requirements.txt), database initialisation, migration execution, and room seeding (seed_rooms.py).
- API Reference: Documentation of any JWT-protected API endpoints for future integration, including request/response formats.
- README File: A concise README in the project root covering quick-start instructions, project structure, and configuration.
- Inline Code Comments: All major functions and route handlers are documented with docstrings explaining their purpose, parameters, and return values.

6. References

- Flask Documentation (v2.3.3). Pallets Projects. https://flask.palletsprojects.com/
- SQLAlchemy ORM Documentation (v2.0). https://docs.sqlalchemy.org/en/20/
- Flask-Login Documentation (v0.6.2). https://flask-login.readthedocs.io/
- Flask-WTF and WTForms Documentation. https://flask-wtf.readthedocs.io/
- Flask-Migrate / Alembic Documentation. https://flask-migrate.readthedocs.io/
- Flask-JWT-Extended Documentation. https://flask-jwt-extended.readthedocs.io/
- IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications. IEEE.
- Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education.
- Pressman, R.S. (2014). Software Engineering: A Practitioner's Approach (8th ed.). McGraw-Hill.

7. Appendices

Appendix A: Database Entity Summary

The following entities are defined in the SQLAlchemy models (app/models.py):

Entity	Key Fields	Primary Relationships
User	id, username, email, password_hash, role	One-to-one with Doctor, Nurse, Receptionist, StoreManager
Patient	id, first_name, last_name, dob, gender, blood_group, contact	Has many Appointments, MedicalRecords, Prescriptions, Admissions
Doctor	id, user_id, specialization, qualification, office_room	Has many Appointments, Prescriptions
Nurse	id, user_id, department	Has many NurseAssignments, PatientStatus updates
Appointment	id, patient_id, doctor_id, date, status, is_inpatient	Linked to Room and Nurse for inpatient; to Bill for billing
Admission	id, patient_id, room_id, doctor_id, nurse_id, status	Linked to Room, Bill; drives inpatient workflow
Room	id, room_number, room_type, status, daily_rate	Referenced by Admissions and Appointments
Prescription	id, patient_id, doctor_id, date, notes	Has many PrescriptionMedication items
Medication	id, name, unit_price, stock_quantity	Referenced by PrescriptionMedication; stock is deducted on dispense
Inventory	id, item_name, category, stock_quantity, reorder_level	Referenced by MedicalProcedure; linked to Supplier
Bill	id, admission_id / appointment_id, room_charges, medication_charges, total	Generated per Admission or Appointment
Order	id, supplier_id, status, total_amount	Has many OrderItems; status: pending/delivered/cancelled

Appendix B: Role-to-Route Mapping

Role	Blueprint	Key Routes
Doctor	doctors	/doctors/dashboard, /doctors/appointments, /doctors/patients, /doctors/prescriptions/new/<id>
Nurse	nurses	/nurses/dashboard, /nurses/patients, /nurses/assignments/new, /nurses/patients/<id>/update_status

Role	Blueprint	Key Routes
Receptionist	receptionists	/receptionists/dashboard, /receptionists/patients, /receptionists/patients/<id>/admit, /receptionists/admissions/<id>/bill
Store Manager	store	/store/dashboard, /store/inventory, /store/suppliers, /store/orders
Patient	patients	/patients, /patients/<id>, /patients/<id>/appointments